

Chapitre 6

Régression linéaire univariée et multivariée

Sommaire

6.1 Régression linéaire univariée et méthode de descente du gradient	81
6.1.1 Construction d'une fonction de coût	83
6.1.2 Minimisation de la fonction coût	84
6.2 Régression linéaire multivariée et écriture matricielle de la méthode de descente du gradient	86
6.3 Fonction hypothèse	87
6.4 Fonction de coût	87
6.5 Minimisation de la fonction de coût par la méthode de gradient	88
6.6 Etape de normalisation	88
6.6.1 Normalisation unitaire	89
6.6.2 Normalisation "centrage réduction"	89
6.6.3 Modules de normalisation de la bibliothèque <code>scikit-learn</code>	89
6.7 Ecriture matricielle de la méthode de descente de gradient	90
6.7.1 Formulation matricielle de la fonction hypothèse	90
6.7.2 Formulation matricielle de la fonction de coût	91
6.7.3 Formulation matricielle de la méthode de descente du gradient	91
6.7.4 Implémentation de la formulation matricielle en Python	92

6.1 Régression linéaire univariée et méthode de descente du gradient

Un calcul de régression consiste à trouver une fonction, appelée modèle ou hypothèse, qui permet d'établir une relation particulière entre des données.

Ces données sont séparées en deux parties : les entrées et la (les) cible(s).

Par exemple, on peut disposer d'un ensemble de données sur des logements pour lesquels sont connus : le nombre de pièces, la superficie et le prix. Les données d'entrée d'un calcul de régression sont dans cet exemple, le nombre de pièces et la superficie. La donnée cible correspond au prix.

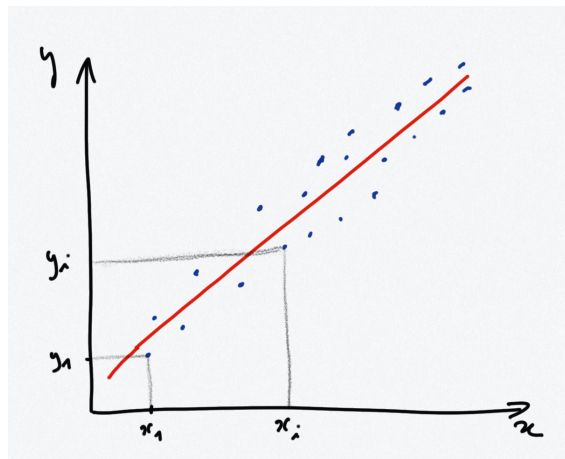
On recherche donc un modèle qui est capable de calculer (ou prédire) le prix d'un logement connaissant sa superficie et son nombre de pièces. Ce modèle doit alors permettre de retrouver un prix identique ou voisin de chacun des logements présents dans les données initiales (on parle de données d'entraînement du modèle) afin ensuite d'être capable de prédire le prix d'un logement, non présent dans les données initiales, à partir de son nombre de pièces et de sa superficie.

Plus généralement, dans un problème de regression, on part d'un ensemble de données et on cherche une expression, c'est à dire une fonction ayant pour variables les données d'entrée et qui calcule une sortie correspondant à la donnée cible, en s'y approchant "au mieux".

On est dans la catégorie de l'apprentissage dit "supervisé" car on va définir a priori la nature de la fonction recherchée. Pour s'approcher "au mieux" de la cible des données, on va agir sur les valeurs d'un certain nombre de paramètres qui caractérisent la fonction recherchée.

Dans la suite, la fonction modèle ou hypothèse est supposée être une fonction linéaire.

A partir d'un ensemble de données $\{(x_i, y_i), 1 \leq i \leq n\}$, on recherche la droite qui passe "au plus près" de l'ensemble de ces points.



La fonction recherchée est linéaire :

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Elle dépend de deux paramètres θ_0 et θ_1 qui caractérisent cette fonction linéaire. Le problème consiste alors à calculer les valeurs de ces deux paramètres pour que la fonction h_{θ} passe "au plus près" des données.

Remarquons déjà que cette fonction h_{θ} n'a qu'une variable. Dans la terminologie usuelle de l'analyse des données, on dira que l'on a une seule variable *explicative* et donc un modèle *univarié* par opposition aux modèles multivariés qui utilisent des fonctions de plusieurs variables.

Pour identifier la fonction h_{θ} , on dispose de données correspondant aux deux vecteurs

$$x = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} \quad y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

On recherche alors la fonction de régression - en analyse de données, on parle de *fonction hypothèse* - qui prend pour variable le vecteur x et qui s'écrit sous la forme

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

On parle du vecteur y comme une *cible* de la fonction hypothèse h . Cela veut dire que l'on cherche à

comparer

$$h_{\theta}(x) = \begin{pmatrix} h_{\theta}(x_1) \\ \vdots \\ h_{\theta}(x_n) \end{pmatrix} \quad \text{avec} \quad y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

Le problème posé est donc de trouver les “meilleures” valeurs des deux paramètres θ_0 et θ_1 que l’on range dans un vecteur θ :

$$\theta = \begin{pmatrix} \theta_0 \\ \theta_1 \end{pmatrix}$$

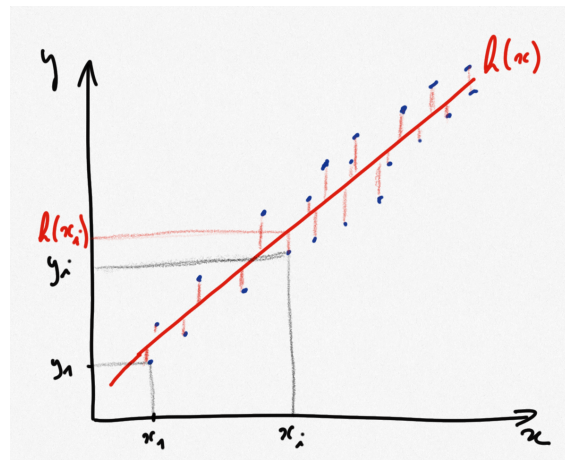
et qui permettent que $h_{\theta}(x)$ soit “le plus proche” de la donnée cible y . Pour cela, on va introduire la notion de *fonction de coût*.

6.1.1 Construction d’une fonction de coût

La question est la suivante : comment formaliser que le vecteur $h_{\theta}(x)$ est “le plus proche” du vecteur y ? Soit la composante i de chacun de ces deux vecteurs : $h_{\theta}(x_i)$ et y_i , on calcule l’écart entre ces deux valeurs par

$$(h_{\theta}(x_i) - y_i)^2$$

Il s’agit du carré des longueurs des segments rouges verticaux sur la figure suivante. Ces longueurs de segments permettent (parmi d’autres méthodes) de mesurer l’écart entre les points et la droite $y = h_{\theta}(x)$. L’élévation au carré évite que ces écarts soient de différents signes et se compensent lorsque l’on va les additionner ou cumuler ; il permet aussi de pénaliser le calcul pour les points très éloignés de la droite.

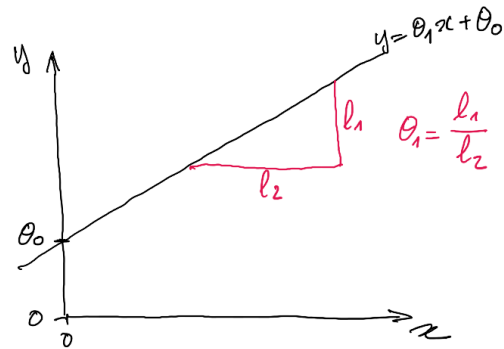


On va alors cumuler (additionner) tous ces écarts en définissant une fonction de coût, appelée **critère des moindres carrés**, dont la variable est le vecteur θ , de composantes θ_0 et θ_1 , paramètres de la fonction hypothèse :

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i)^2$$

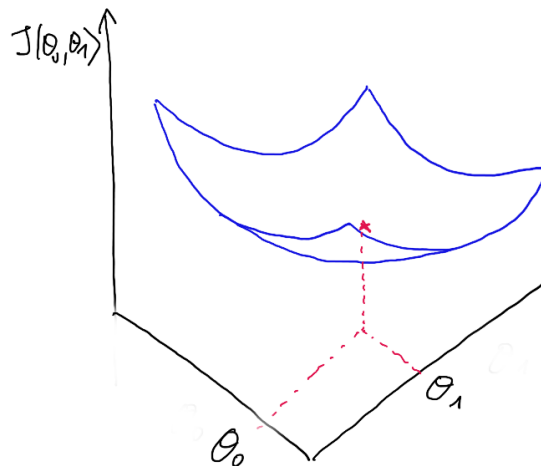
$$J(\theta) = J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (\theta_0 + \theta_1 x_i - y_i)^2$$

Rappel : $y = h_{\theta}(x) = \theta_0 + \theta_1 x$ est l’équation d’une droite que l’on représente sur la figure suivante et dont le coefficient directeur (appelé aussi pente de la droite) vaut $\theta_1 = \frac{l_1}{l_2}$



6.1.2 Minimisation de la fonction coût

On cherche donc θ_0 et θ_1 qui minimisent $J(\theta_0, \theta_1)$

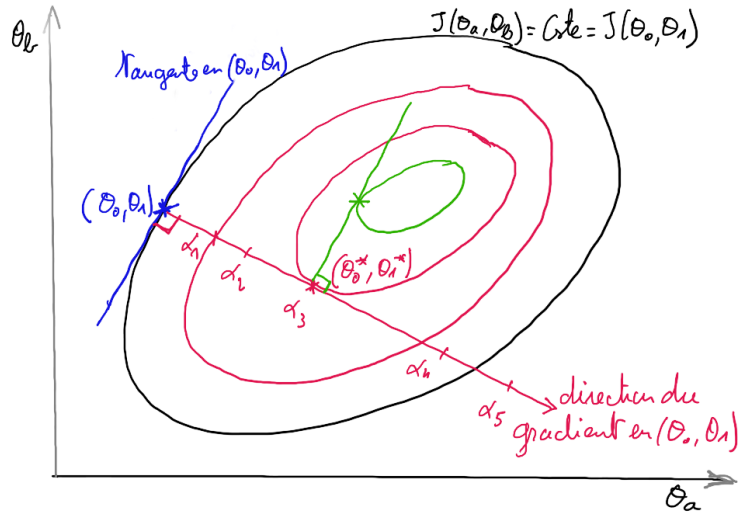


Pour cela, on va utiliser une méthode itérative, la **méthode de descente du gradient**.

On définit tout d'abord le vecteur gradient de J , noté $\nabla J(\theta)$ par

$$\begin{pmatrix} \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) \\ \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) \end{pmatrix}$$

On montre que $\nabla J(\theta)$ est un vecteur orthogonal aux tangentes des courbes de niveau d'équation $J(\theta_0, \theta_1) = Cste$, comme cela est représenté sur la figure suivante.



La méthode itérative de descente du gradient est la suivante :

On démarre avec une valeur initiale du vecteur θ de coordonnées (θ_0, θ_1) et on recherche un nouveau vecteur, notée θ^* de coordonnées (θ_0^*, θ_1^*) tel que

$$J(\theta_0^*, \theta_1^*) < J(\theta_0, \theta_1)$$

θ^* s'obtient à partir de θ en suivant une *direction de descente* portée par $\nabla J(\theta)$, c'est à dire :

$$\theta^* = \theta - \alpha \nabla J(\theta)$$

où α est un paramètre que l'on va définir plus tard.

L'expression précédente du vecteur θ^* s'écrit pour chacune de ses deux composantes :

$$\begin{cases} \theta_0^* = \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) \\ \theta_1^* = \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) \end{cases}$$

Le calcul des dérivées partielles se fait de la manière suivante

$$\frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \frac{\partial}{\partial \theta_0} \left(\frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i)^2 \right) \quad (6.1)$$

$$= \frac{\partial}{\partial \theta_0} \left(\frac{1}{2m} \sum_{i=1}^m (\theta_0 + \theta_1 x_i - y_i)^2 \right) \quad (6.2)$$

$$= \frac{1}{2m} \sum_{i=1}^m \frac{\partial}{\partial \theta_0} ((\theta_0 + \theta_1 x_i - y_i)^2) \quad (6.3)$$

$$= \frac{1}{2m} \sum_{i=1}^m 2(\theta_0 + \theta_1 x_i - y_i) \frac{\partial}{\partial \theta_0} (\theta_0 + \theta_1 x_i - y_i) \quad (6.4)$$

$$= \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i) \quad (6.5)$$

De la même manière

$$\frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m 2(\theta_0 + \theta_1 x_i - y_i) \frac{\partial}{\partial \theta_1} ((\theta_0 + \theta_1 x_i - y_i)) \quad (6.6)$$

$$= \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i) x_i \quad (6.7)$$

La formule d'itération précédente s'écrit alors

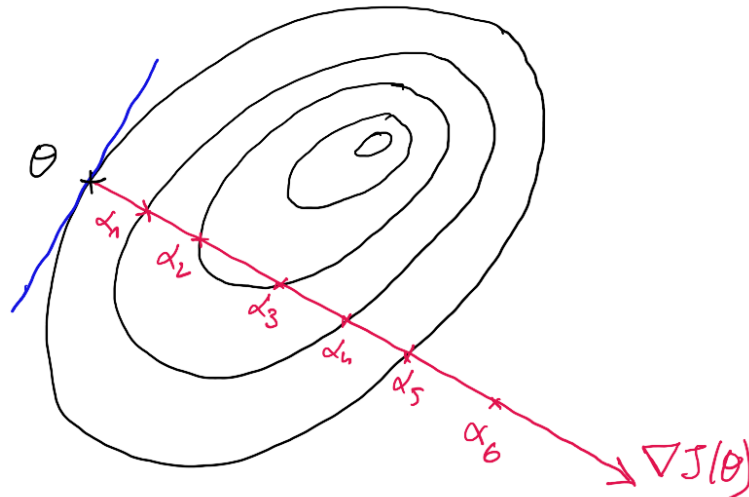
$$\begin{cases} \theta_0^* = \theta_0 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i) \\ \theta_1^* = \theta_1 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i) x_i \end{cases}$$

Avant de passer à l'itération suivante, on remplacera θ_0 par θ_0^* et θ_1 par θ_1^*

Choix de α

La question qui se pose est de savoir choisir α que l'on va appeler *coefficient d'apprentissage* en science des données. Ce coefficient règle la profondeur de descente à chaque itération.

On peut essayer d'“optimiser” sa valeur en effectuant des séries de tests successifs. En effet, on a intérêt à prendre α assez grand comme l'illustre la figure suivant. Mais on voit aussi que si α est trop grand, on risque de “rater” le minimum.



6.2 Régression linéaire multivariée et écriture matricielle de la méthode de descente du gradient

Nous nous intéressons ici à une extension du modèle univarié (à un degré de liberté) à un modèle à plusieurs degrés de liberté (ici, n) qui correspondent à un ensemble de m données d'entrée à n composantes

(x_{ij}) et qui forment donc une matrice de dimension (m, n) :

$$X = \begin{pmatrix} x_{11} & \dots & x_{1n} \\ \vdots & & \vdots \\ x_{m1} & \dots & x_{mn} \end{pmatrix}$$

Les données sont toujours constituées d'une valeur cible (y_i) et l'ensemble des données est décrit donc par

$$\{(x_{ij}, y_i) ; \quad 1 \leq j \leq n \quad \text{et} \quad 1 \leq i \leq m\}$$

Nous considérons donc que dans un modèle multivarié, nous avons n observations plutôt qu'une seule dans le modèle univarié. Chaque observation correspond à une colonne de la matrice X précédente. On note X_i la colonne i de la matrice.

6.3 Fonction hypothèse

L'hypothèse (ou fonction de régression) prend en entrée une matrice et calcule un vecteur :

$$h_\theta(X) = \theta_0 I_1 + \theta_1 X_1 + \theta_2 X_2 + \dots + \theta_n X_n$$

où l'on note I_1 le vecteur dont toutes les m composantes sont égales à 1.

Cette expression est linéaire par rapport aux observations X_i .

On note x_i le vecteur contenant la donnée i composée de n observations (attention à l'usage des majuscules et minuscules !).

$x_i = i^{eme}$ ligne de la matrice X

$$x_i = (x_{i1} \ x_{i2} \ \dots \ x_{in})$$

La projection de h_θ sur chacun de ces vecteurs x_i s'écrit donc :

$$h_\theta(x_i) = \theta_0 + \theta_1 x_{i1} + \theta_2 x_{i2} + \dots + \theta_n x_{in}$$

Le problème de régression linéaire multivariée consiste ainsi à trouver le vecteur

$$\theta = \begin{pmatrix} \theta_0 \\ \vdots \\ \theta_n \end{pmatrix}$$

tel que $h_\theta(X)$ soit "le plus proche" de y .

6.4 Fonction de coût

La fonction de coût utilisée correspond au critère des moindres carrés utilisé dans le chapitre sur la régression linéaire univariée :

$$\begin{aligned} J(\theta) &= J(\theta_0, \theta_1, \dots, \theta_n) \\ &= \frac{1}{2m} \sum_{i=1}^m (h_\theta(x_i) - y_i)^2 \end{aligned}$$

6.5 Minimisation de la fonction de coût par la méthode de gradient

Résoudre le problème de régression linéaire multivarié consiste à chercher $\theta = (\theta_0, \theta_1, \dots, \theta_n)$ qui rend $J(\theta)$ minimal. Le processus décrit pour la régression linéaire univariée se généralise assez simplement comme cela est décrit ci-après.

A partir de valeurs initiales (arbitraires) des composantes de $\theta = (\theta_0, \theta_1, \dots, \theta_n)$, une méthode itérative est utilisée.

Description d'une itération

A partir de $\theta = (\theta_0, \theta_1, \dots, \theta_n)$ on va calculer $\theta^* = (\theta_0^*, \theta_1^*, \dots, \theta_n^*)$ tel que

$$J(\theta^*) < J(\theta)$$

θ^* se calcule à partir du gradient de la fonction J , noté $\nabla J(\theta)$, grâce à la formule :

$$\theta^* = \theta - \alpha \nabla J(\theta)$$

$$\begin{cases} \theta_0^* &= \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta) \\ &\vdots \\ \theta_j^* &= \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \\ &\vdots \\ \theta_n^* &= \theta_n - \alpha \frac{\partial}{\partial \theta_n} J(\theta) \end{cases}$$

Le calcul des dérivées partielles a été expliqué dans la partie A du chapitre portant sur la régression linéaire univarié. Il n'est pas décrit ici mais sera repris dans la section suivante.

Avant de passer à l'itération suivante, on va recopier le vecteur θ^* dans θ .

Ce schéma itératif est exécuté un nombre fini de fois qui peut être calculé automatiquement lorsque la différence relative entre θ^* et θ est faible, c'est à dire :

$$\frac{\|\theta^* - \theta\|}{\|\theta\|} < \epsilon$$

pour ϵ une “petite” valeur.

6.6 Etape de normalisation

Dans les données d'entrée représentées par la matrice X , il est possible que des observations aient des ordres de grandeur différents. Par exemple, prenons le problème “classique” d'estimation du prix d'un appartement en fonction de deux observations : le nombre de pièces et sa superficie. Dans un jeu de données, on va supposer que le nombre de pièces varie entre 1 et 9 alors que la superficie varie entre 20 et 200 m². Du fait de la différence d'ordre de grandeur entre ces 2 observations, l'algorithme de descente du gradient aura du mal à converger.

Pour pallier à cette difficulté, on va “normaliser” chaque observation, c'est à dire chaque vecteur colonne X_i de la matrice X , de sorte que ces différentes observations soient comparables et du même ordre de grandeur.

Plusieurs méthodes de normalisation sont possibles. Nous en présentons deux.

6.6.1 Normalisation unitaire

On normalise chaque observation X_i pour que ses coefficients soient compris entre 0 et 1.

Soit X_{ij} la j^{eme} composante du vecteur X_i , soit $\max(X_i)$ la plus grande composante du vecteur X_i et $\min(X_i)$ la plus petite, on utilise la formule suivante pour calculer $\overline{X_{ij}}$ la valeur normalisée de X_{ij} :

$$\overline{X_{ij}} = \frac{X_{ij} - \min(X_i)}{\max(X_i) - \min(X_i)}$$

6.6.2 Normalisation “centrage réduction”

Plutôt que d’imposer un intervalle dans lequel s’inscrivent les coefficients, on va contrôler la dispersion des valeurs en ramenant l’écart-type à 1, et la moyenne à 0.

Rappels : Soit Z un vecteur de valeurs de composantes z_i , pour $1 \leq i \leq n$. On définit la moyenne par :

$$\mu(Z) = \frac{1}{n} \sum_{i=1}^n z_i$$

L’écart-type, racine carrée de la variance, permet de mesurer la dispersion des données autour de la moyenne. Il se calcule par :

$$\sigma(Z) = \sqrt{V(Z)} = \sqrt{\frac{1}{n} \sum_{i=1}^n (z_i - \mu(Z))^2}$$

La normalisation “centrage réduction” des données de l’observation X_i se calcule par

$$\overline{X_{ij}} = \frac{X_{ij} - \mu(X_i)}{\sigma(X_i)}$$

Ainsi la moyenne des valeurs normalisées vaut 0 et son écart-type vaut 1.

6.6.3 Modules de normalisation de la bibliothèque `scikit-learn`

La bibliothèque `scikit-learn` de Python, offre plusieurs méthodes permettant de normaliser les données. Cette normalisation est donc une étape importante pour “préparer” les jeux de données avant leur traitement. Nous donnons ci-après les deux modules de normalisation de `scikit-learn` (parmi d’autres) qui réalisent les 2 méthodes de normalisations décrites précédemment :

- `sklearn.preprocessing.MinMaxScaler` est le module de normalisation qui implémente la formule du paragraphe 4.1
- `sklearn.preprocessing.StandardScaler` est le module de normalisation qui implémente la formule du paragraphe 4.2

Voici un exemple d’utilisation

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler
scaler = StandardScaler()
# house correspond aux données sur les appartements
new_house = scaler.fit_transform(house)
```

6.7 Ecriture matricielle de la méthode de descente de gradient

6.7.1 Formulation matricielle de la fonction hypothèse

En reprenant les notations introduites dans les sections précédentes, la fonction hypothèse s'écrit

$$h_{\theta}(X) = \theta_0 I_1 + \theta_1 X_1 + \theta_2 X_2 + \cdots + \theta_n X_n$$

où l'on note I_1 le vecteur dont toutes les m composantes sont égales à 1.

Pour chaque donnée i dont les valeurs des n observations sont dans le vecteur x_i , l'hypothèse se projette de la manière suivante :

$$h_{\theta}(x_i) = \theta_0 + \theta_1 x_{i1} + \theta_2 x_{i2} + \cdots + \theta_n x_{in}$$

A la matrice de données d'entrée X on ajoute en première colonne le vecteur I_1 et la matrice X sera donc maintenant :

$$X = \begin{bmatrix} 1 & x_{11} & \cdots & x_{1n} \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_{m1} & \cdots & x_{mn} \end{bmatrix}$$

Et la fonction hypothèse se calcule comme le produit matriciel :

$$h_{\theta}(X) = X\theta$$

où θ correspond au vecteur défini précédemment :

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}$$

En effet

$$X\theta = \begin{bmatrix} 1 & x_{11} & \cdots & x_{1n} \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_{m1} & \cdots & x_{mn} \end{bmatrix} \begin{bmatrix} \theta_0 \\ \vdots \\ \theta_n \end{bmatrix}$$

$$X\theta = \begin{bmatrix} \theta_0 + \theta_1 x_{11} + \cdots + \theta_n x_{1n} \\ \vdots \\ \theta_0 + \theta_1 x_{in} + \cdots + \theta_n x_{in} \\ \vdots \\ \theta_0 + \theta_1 x_{m1} + \cdots + \theta_n x_{mn} \end{bmatrix}$$

On retrouve bien que l'expression du i^{eme} coefficient de ce vecteur est celle de $h_{\theta}(x_i)$

6.7.2 Formulation matricielle de la fonction de coût

La fonction de coût est une fonction de $\mathbb{R}^2$ dans $\mathbb{R}$; elle s'écrit

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i)^2$$

Soit le vecteur de données cible

$$Y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

On pose le vecteur

$$E = h_{\theta}(X) - Y$$

$$E = \begin{bmatrix} h_{\theta}(x_1) \\ \vdots \\ h_{\theta}(x_n) \end{bmatrix} - \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

$$E = \begin{bmatrix} h_{\theta}(x_1) - y_1 \\ \vdots \\ h_{\theta}(x_n) - y_n \end{bmatrix}$$

Sachant que le carré scalaire d'un vecteur Z , de composantes z_i s'écrit

$$Z^t Z = \sum_{i=1}^m (z_i)^2$$

Le carré scalaire du vecteur E s'écrit alors

$$E^t E = \sum_{i=1}^m (h_{\theta}(x_i) - y_i)^2$$

et la fonction de coût s'écrit alors sous forme matricielle

$$J(\theta) = \frac{1}{2m} E^t E$$

6.7.3 Formulation matricielle de la méthode de descente du gradient

La méthode de descente du gradient consiste à appliquer des itérations successives qui prennent en entrée un vecteur θ pour calculer une nouvelle valeur de ce vecteur θ^* tel que $J(\theta^*) < J(\theta)$.

Cette nouvelle valeur θ^* sert alors d'entrée pour la nouvelle itération. Pour initier le processus, on choisit arbitrairement la valeur initiale de θ .

On peut choisir de s'arrêter après un nombre arbitraire d'itérations ou alors contrôler la convergence du calcul de θ et prendre un critère d'arrêt qui peut s'écrire :

$$\left| \frac{J(\theta^*) - J(\theta)}{J(\theta)} \right| < \epsilon$$

où ϵ est une valeur petite de l'ordre de 10^{-3} ou plus petit.

Le processus itératif dans une régression linéaire multivariée est une généralisation du processus utilisé pour une régression linéaire univariée :

$$\begin{cases} \theta_0^* = \theta_0 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i) \\ \theta_1^* = \theta_1 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i) x_{i1} \\ \vdots \\ \theta_j^* = \theta_j - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i) x_{ij} \\ \vdots \\ \theta_n^* = \theta_n - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i) x_{in} \end{cases}$$

Ce processus s'écrit sous la forme matricielle suivante :

$$\theta^* = \theta - \frac{\alpha}{m} X^t E$$

En effet

$$X^t E = \begin{bmatrix} 1 & \dots & 1 \\ x_{11} & \dots & x_{m1} \\ \vdots & & \\ x_{1n} & \dots & x_{mn} \end{bmatrix} \begin{bmatrix} h_{\theta}(x_1) - y_1 \\ \vdots \\ h_{\theta}(x_m) - y_m \end{bmatrix}$$

$$X^t E = \begin{bmatrix} \sum_{i=1}^m (h_{\theta}(x_i) - y_i) \\ \sum_{i=1}^m (h_{\theta}(x_i) - y_i) x_{i1} \\ \vdots \\ \sum_{i=1}^m (h_{\theta}(x_i) - y_i) x_{ij} \\ \vdots \\ \sum_{i=1}^m (h_{\theta}(x_i) - y_i) x_{in} \end{bmatrix}$$

6.7.4 Implémentation de la formulation matricielle en Python

1. On lit les données correspondant à n observations en entrée et une valeur de sortie

$$\begin{bmatrix} x_{11} & \dots & x_{1n} \\ \vdots & & \vdots \\ x_{m1} & \dots & x_{mn} \end{bmatrix} \quad \text{et} \quad \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

2. On construit la matrice X augmentée et le vecteur Y

$$X = \begin{bmatrix} 1 & x_{11} & \dots & x_{1n} \\ \vdots & \vdots & & \vdots \\ 1 & x_{m1} & \dots & x_{mn} \end{bmatrix}$$

$$Y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

3. On définit la fonction hypothèse ayant pour variables, le vecteur θ et la matrice augmentée X et renvoyant le vecteur :

$$h(\theta, X) = X\theta$$

4. On définit la fonction de coût, de variable θ :

$$J(\theta) = \frac{1}{2m} E^t E$$

où $E = h_\theta(X) - Y$

5. On définit une fonction itération de la méthode de descente du gradient, qui prend en entrée θ et qui calcule θ^* par la formule :

$$\theta^* = \theta - \frac{\alpha}{m} X^t E$$

On fixe alors un nombre donné d'itérations en partant d'une valeur arbitraire de θ ou alors on contrôle la convergence de calcul de θ avec le critère décrit précédemment.

TP Régression linéaire

Exercice 1 - Régression linéaire univarié

Les données utilisées pour cet exercice décrivent les tailles d'enfants d'âge compris entre 2 et 8 ans : le fichier `ex1x.dat` correspond à leur âge et `ex1y.dat` correspond à leur taille (en mètres). On a dans ces 2 fichiers, les données de 50 enfants rangées dans le même ordre.

Ces données constituent des exemples d'apprentissage qui vont être utilisées afin de construire un modèle de régression linéaire qui a pour objectif de prédire la taille d'un enfant à partir de son âge.

1. Tout d'abord, nous allons utiliser la fonction `loadtxt` du package `numpy` en Python (ici désigné par `np`). Cette fonction permet de lire les fichiers et de les convertir en vecteurs/matrices :

```
x=np.loadtxt('ex1x.dat');  
y=np.loadtxt('ex1y.dat');
```

2. Afficher le nuage de points (x_i, y_i) .
3. Définir en Python, la fonction hypothèse correspondant à un modèle de régression linéaire. Soit le vecteur $\theta = (\theta_0, \theta_1)$, cette fonction hypothèse s'écrit

$$h(\theta_0, \theta_1, x) = \theta_0 + \theta_1 x$$

4. Définir en Python, la fonction de coût $J(\theta_0, \theta_1)$ telle que vue en cours.
5. Définir en Python, une fonction qui réalise une itération et qui va renvoyer θ_0^* et θ_1^* , en fonction de θ_0 et θ_1 , selon les formules de descente de gradient vue en cours.
Pour faire fonctionner cette méthode de gradient, il faut définir la valeur du coefficient d'apprentissage noté α dans le cours ; il règle la profondeur de descente. On propose ici que ce coefficient prenne une valeur constante égale à 0,07. On partira aussi des valeurs initiales $\theta_0 = \theta_1 = 0$.
6. Faire tourner la méthode de descente de gradient sur quelques itérations puis représenter la droite

$$y = \theta_0 + \theta_1 x$$

sur le nuage de points.

7. Faire tourner la méthode de descente de gradient pendant toutes les itérations nécessaires à faire converger la solution θ recherchée. Pour cela on définit le critère d'arrêt du processus itératif par

$$\left| \frac{J(\theta^*) - J(\theta)}{J(\theta)} \right| < 10^{-3}$$

Afficher la droite obtenue suite à cette convergence sur le nuage de points.

8. On peut désormais utiliser le modèle pour faire des prédictions : quelle est la taille de 3 enfants d'âges respectifs 3, 5 et 7 ans ?
9. Nous allons maintenant visualiser la fonction de coût $J(\theta)$ en 3D sur une grille de base 100 x 100 et des valeurs pour θ dans les intervalles suivants (il faudra donc prendre 100 valeurs réparties dans ces intervalles) : $\theta_0 \in [-30; 30]$ et $\theta_1 \in [-3; 3]$.

Exercice 2 - Régression linéaire multivariée et écriture matricielle de la méthode de descente du gradient

Dans ce problème, on utilise des données correspondant à 47 exemples d'apprentissage sur des données immobilières à Portland, Oregon (USA). Les données d'entrée x (stockées dans le fichier `ex2x.dat`) correspondent aux surfaces et au nombre de pièces de ces 47 appartements et la donnée cible y (stockée dans le fichier `ex2y.dat`) correspond au prix de ces mêmes appartements.

1. Utiliser la fonction `loadtxt` du package `numpy` en Python (ici désigné par `np`). Cette fonction permet de lire les fichiers et de les convertir en vecteurs/matrices :


```
x=np.loadtxt('ex2x.dat');
y=np.loadtxt('ex2y.dat');
```
2. Prétraitement des données : utiliser la fonction `sklearn.preprocessing.StandardScaler` pour normaliser les données.
3. Définir les fonctions en Python sous forme matricielle pour représenter la fonction hypothèse $h_\theta(X)$, le vecteur tel que défini dans le cours : $E = h_\theta(X) - Y$, la fonction décrivant une itération et la fonction de coût $J(\theta)$.
4. Pour la valeur du taux d'apprentissage $\alpha = 0,07$, effectuer le calcul de régression permettant d'obtenir le vecteur $\theta = (\theta_0, \theta_1, \theta_2)$ optimal permettant de calculer la meilleure régression linéaire multivariée sur le jeu de données d'apprentissage.
5. Nous allons maintenant automatiser la recherche du meilleur taux d'apprentissage $\alpha \in [0,001; 10]$. Pour ce faire, on devra calculer pour chaque itération la valeur de la fonction de coût $J(\theta)$ et on stockera toutes ces valeurs dans un vecteur. Comme on veut sélectionner un taux d'apprentissage efficace, on va comparer les résultats de calcul de $J(\theta)$ sur 50 itérations en changeant de taux d'apprentissage à chaque série d'itérations. Les valeurs de ce taux doivent rester dans l'intervalle initialement donné $\alpha \in [0,001; 10]$. On tracera alors les courbes représentant en abscisse, le nombre d'itérations et en ordonnées les valeurs de la fonction de coût ; chaque courbe correspond à une valeur du taux d'apprentissage. A partir de ces courbes, sélectionner ce qui paraît être le meilleur taux d'apprentissage et recalculer le vecteur θ jusqu'à la convergence. Utiliser ce vecteur θ pour prédire le prix d'un logement de 1650 m² et de 3 pièces.